Arrays

Mostafa Faik

September 1, 2024

Introduction

In this lab, we explored the efficiency of different operations over an array of elements, focusing on performance measurement. Our goal was to benchmark three operations: `random access`, `searching through an array`, and `finding duplicates between two arrays` to understand how execution time scales with the size of the array. These operations were analyzed with respect to their time complexity, particularly focusing on $O(1)$, $O(n)$, and $O(n^2)$ behavior.

Benchmarking operations

- Random access:

- This operation involves reading or writing a value at a random location in an array.
- This operation is expected to be constant time, i.e., $O(1)$, because it's an instantaneous operation when accessing an element at any index in an array.

- Search:

- This operation is used to traverse an array to find a value/item.
- This operation is expected to be $O(n)$, because we may need to inspect each element in the worst case when searching in an unsorted array.

- Finding duplicates:

- This operation checks two arrays to identify common elements.
- The operation of finding duplicates by comparing every element in the first array with every element in the second array is expected to be $O(n^2)$, where n is the length of the arrays.

Checking the accuracy of the clock

We first checked the accuracy of `System.nanoTime()` in telling the time it takes to do a specific task. This was done by running the following code that was provided in the assignment.

```
for (int i = 0; i < 10; i++) {
      long n0 = System.nanoTime();
      long n1 = System.nanoTime();
      System.out.println(" resolution " + (n1 - n0) + " nanoseconds");
  }
```

When running the code different times, I observed a typical resolution of around $100ns$, with occasional readings showing $0ns$. This is likely due to the clock's resolution limits, the system's concurrent processes or the system clock couldn't detect any time passing between the two calls. This can happen if the calls to `System.nanoTime()` are faster than the clock's resolution. This behavior can also be because of the fact that the operation is being done on a computer with other background tasks running. These inconsistencies highlight the need for averaging multiple measurements to obtain a reliable benchmark data and the need for a better benchmarking process.

Random access

In this task we investigated the efficiency of array operations, focusing on measuring the access time for array elements. We implemented a benchmark that measured the time taken to perform random access operations on arrays of various sizes. We tested arrays of sizes with different elements. For each size, we performed multiple benchmarks to determine the minimum access time. We also calculated average and maximum times for each benchmark iteration. The following results were obtained after performing the benchmarks:

Case	n	Time
1	100	2.1 ns
2	200	2.3 ns
3	400	2.4 ns
4	800	2.3 ns
5	1600	2.8 ns
6	3200	3.7 ns

Table 1: Random Access Time Measurements

As we can see from the table above, the results were fairly consistent after implementing the JIT warm-up, indicating that the measurements were reliable and the JIT compiler optimizations were effective. There was a slight increase in access time as array size increases, but the increase was minimal. Larger arrays showed higher access times, possibly due to cache effects. But the increase was consistent and didn't affect performance.

Searching

In this task we used the benchmarking system from the previous task and tried to find the time taken for an algorithm which searches for a key element in an array. We created an array *array* of size n filled with random integers. We also generated an array *keys* of size *loop* with random integers. For each key in *keys* we searched through the entire *array* to check if the key exists. We tested various array sizes n and recorded the execution time for searching through *loop* keys. This was the result that i got:

Case	**n**	**Time**
1	100	19.8 ns
2	200	33.6 ns
3	400	62.0 ns
4	800	117.0 ns
5	1600	225.4 ns
6	3200	438.1 ns

Table 2: Array Search Benchmark Results

We observe that the time complexity of searching through an unsorted array is linear with respect to the size of the array. Each search operation has to potentially check every element in the array, leading to a time complexity of $O(n)$. The execution time $T(n)$ can be approximated by a linear polynomial in relation to the size of the array:

$$T(n) = kn$$

Where k is a constant representing the time per search operation. This linear relationship is expected due to the need to potentially check each element in the array to find the target key.

Finding duplicates in two arrays

In this task, we extend the concepts from the previous benchmarking exercises to measure the time taken to search for duplicates between two arrays.

Each array had the same number of randomly generated keys, and we aimed to identify how the execution time scaled with the array size. The benchmark in this implementation was that for each element in *array a*, the entire *array b* was searched to find a match. The time taken for the duplicate search was then measured using `System.nanoTime()`. The results for finding duplicates was as follows:

Case	**n**	**Time**
1	100	19.0 ns
2	200	53.4 ns
3	400	82.6 ns
4	800	330.2 ns
5	1600	1321 ns
6	3200	5286 ns

Table 3: Duplicate Search Time Measurements

We observe that for very small array sizes, the measured times show some inconsistency, making timing results unreliable for very small sizes. For larger sizes, as the array size increases the measured times become more consistent across different runs, suggesting that the benchmark's reliability improves with larger input sizes.

We can conclude, that the time complexity of this operation is $O(n^2)$. This is because for each element in *array a*, we perform a linear search through *array b*, leading to a quadratic growth in execution time as the size of the array increases. The time taken approximately follows a quadratic polynomial:

$$T(n) = an^2 + bn + c$$

where a, b and c are constants. The increase in time with increasing n is expected due to the nested loop structure of the search algorithm.

Conclusion

In this lab, we saw how basic array operations perform in real scenarios, confirming their theoretical time complexities. The results show why it's crucial to consider time complexity when designing algorithms, especially with large datasets where small inefficiencies can greatly impact performance. This exercise emphasizes the importance of knowing both the theory and the real world effects of algorithm efficiency.